ΜΑΘΗΜΑ 7 ΑΠΟ 15 · ΥΠΟΕΝΟΤΗΤΑ 2.3.Γ.3

# Αναδρομή και βιβλιοθήκες

Συναρτήσεις που καλούν τον εαυτό τους — και κώδικας που έγραψαν άλλοι

Θεωρία σε 14 μικρά βήματα + 18 ασκήσεις στο εργαστήριο  Διάρκεια: 4 ώρες (3η–6η)  24 διαφάνειες

Προγραμματισμός Εφαρμογών Διαδικτύου & Παιχνιδιών (Python, Pygame) · Γ΄ εξάμηνο · Δευτέρα 16/11/2026

# Τι θα ξέρεις στο τέλος

1. Να γράφεις **αναδρομικές** συναρτήσεις: παραγοντικό, Fibonacci, δυαδική αναζήτηση.
2. Να χρησιμοποιείς έτοιμες βιβλιοθήκες με **import**: math, random, datetime.
3. Να φτιάχνεις **δικά σου modules** και να ξέρεις τι κάνει το **if __name__ == "__main__"**.
4. Να εγκαθιστάς πακέτα με **pip** και να κάνεις πράξεις με **NumPy** και γραφήματα με **Matplotlib**.
5. Να **σχεδιάζεις** ένα πρόγραμμα σε μικρά υποπρογράμματα (τμηματικός προγραμματισμός).

### Πώς δουλεύουμε

Η θεωρία είναι χωρισμένη σε **14 μικρά βήματα**. Κάθε βήμα έχει πάνω δεξιά την ετικέτα **ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ** με τον αριθμό της άσκησης στο site.
**Διαβάζεις τη διαφάνεια → λύνεις την άσκηση → πας στο επόμενο βήμα.**

### Από το εύκολο στο δύσκολο

Ασκήσεις **1–8**: εύκολες, μία ιδέα τη φορά.
Ασκήσεις **9–14**: μέτριες, modules, NumPy, γραφήματα.
Ασκήσεις **15–17**: δύσκολες, όλα μαζί. Η **18** είναι για το σπίτι.

**ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ** Το NumPy και το Matplotlib φορτώνουν **μόνα τους** με το import (την πρώτη φορά θέλει λίγα δευτερόλεπτα).

# Θυμάσαι από το Μάθημα 6;

**ΕΚΦΩΝΗΣΗ**

Πέντε γρήγορες ερωτήσεις, 5 λεπτά. Τι εμφανίζει κάθε γραμμή; Πρώτα σκέψου, μετά δες τη λύση.

**ΛΥΣΗ**

```
def f(x, y=2):
    return x * y

def g(x):
    print(x + 1)
```

| ΚΛΉΣΗ | ΕΜΦΑΝΊΖΕΙ | ΓΙΑΤΊ |
|---|---|---|
| `print(f(3))` | 6 | y = 2 (προεπιλογή) |
| `print(f(3, 10))` | 30 | y = 10 |
| `print(f(y=1, x=5))` | 5 | ονομαστικά |
| `print(f(f(2)))` | 8 | f(4) → 8 |
| `print(g(1))` | 2 · None | **g χωρίς return** |

**Σήμερα**

Μια συνάρτηση μπορεί να καλέσει **άλλη** συνάρτηση — ακόμα και **τον εαυτό της**!

**ΣΥΝΔΕΣΗ** Το **f(f(2))** είναι ήδη μια συνάρτηση μέσα σε συνάρτηση. Η αναδρομή πάει ένα βήμα παραπέρα.

# Δύο ιδέες για τη σημερινή μέρα

**1 · ΑΝΑΔΡΟΜΗ**

```
def countdown(n):
    if n == 0:
        print("Φύγαμε!")
    else:
        print(n)
        countdown(n - 1)    # καλεί τον εαυτό της!
```

## Αναδρομή

Λύνεις ένα πρόβλημα λύνοντας ένα **μικρότερο ίδιο** πρόβλημα, μέχρι να φτάσεις σε κάτι **πολύ απλό**.

**2 · ΒΙΒΛΙΟΘΗΚΕΣ**

```
import numpy as np
import matplotlib.pyplot as plt

grades = np.array([12, 9, 17, 15])
plt.bar(["Α", "Β", "Γ", "Δ"], grades)
plt.show()
```

## Βιβλιοθήκες

Δεν γράφεις τα πάντα από την αρχή: **import** και χρησιμοποιείς κώδικα που γράφτηκε και δοκιμάστηκε από άλλους.

**ΤΑ ΒΗΜΑΤΑ** **1–5**: αναδρομή. **6–10**: import, δικά μας modules, pip. **11–13**: NumPy, Matplotlib. **14**: σχεδιάζω σε υποπρογράμματα.

# Μια συνάρτηση που καλεί τον εαυτό της

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 1**

```
def countdown(n):
    if n == 0:                # η βάση: σταματάω
        print("Φύγαμε!")
    else:
        print(n)
        countdown(n - 1)      # το ίδιο, με μικρότερο n

countdown(3)
3
2
1
Φύγαμε!
```

## Αναδρομή

Η συνάρτηση **καλεί τον εαυτό της** με ένα **μικρότερο** πρόβλημα: countdown(3) → countdown(2) → countdown(1) → countdown(0).

## Η βάση

Η περίπτωση που λύνεται **χωρίς** νέα κλήση: εδώ το **n == 0**. Εκεί η αναδρομή **σταματά**.

## Χωρίς βάση

Η συνάρτηση καλεί τον εαυτό της για πάντα → **RecursionError**. Κάθε αναδρομή χρειάζεται **βάση** και **μικραίνει** προς αυτήν.

ΙΔΙΟ ΜΕ Ένα while που μετράει αντίστροφα. Η αναδρομή δεν κάνει κάτι **νέο** — σκέφτεται το πρόβλημα **αλλιώς**.

# Το παραγοντικό, αναδρομικά

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 2**

## ΑΝΑΔΡΟΜΙΚΑ

```
def factorial(n):
    if n <= 1:                      # βάση
        return 1
    return n * factorial(n - 1)     # αναδρομικό βήμα

print(factorial(5))
120
```

## ΤΟ ΙΔΙΟ ΜΕ for

```
def factorial(n):
    result = 1
    for i in range(2, n + 1):
        result = result * i
    return result
```

## Ο ΟΡΙΣΜΟΣ ΑΠΟ ΤΑ ΜΑΘΗΜΑΤΙΚΑ

| ΤΙ | ΠΩΣ ΓΡΑΦΕΤΑΙ |
|---|---|
| 5! | 5 · 4 · 3 · 2 · 1 = 120 |
| 5! με λόγια | 5 · **4!** |
| γενικά | n! = n · **(n – 1)!** |
| βάση | 1! = 1 (και 0! = 1) |

### Η συνταγή

1. Ποια είναι η **πιο απλή** περίπτωση; → **return** την απάντηση.
2. Αλλιώς: λύσε το **μικρότερο** πρόβλημα με κλήση στον εαυτό σου και **χτίσε** πάνω του.

**ΓΙΑΤΙ return** Το **factorial(n - 1)** πρέπει να **δώσει πίσω** τιμή για να πολλαπλασιαστεί με το n. Χωρίς return → None.

# Πώς τρέχει μια αναδρομή

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 3**

`factorial(4)`: ΚΑΤΕΒΑΙΝΕΙ ↓ ΚΑΙ ΜΕΤΑ ΑΝΕΒΑΙΝΕΙ ↑

| ΚΛΗΣΗ | ΠΕΡΙΜΕΝΕΙ… | ΔΙΝΕΙ ΠΙΣΩ |
|---|---|---|
| `factorial(4)` | `4 * factorial(3)` | `4 * 6 =` **24** |
| `factorial(3)` | `3 * factorial(2)` | `3 * 2 =` **6** |
| `factorial(2)` | `2 * factorial(1)` | `2 * 1 =` **2** |
| `factorial(1)` | — βάση — | **1** |

ΧΩΡΙΣ ΒΑΣΗ

```
def forever(n):
    return forever(n + 1)    # δεν μικραίνει, δεν σταματά

forever(1)
RecursionError: maximum recursion depth exceeded
```

### Κατεβαίνει

Κάθε κλήση **περιμένει** την επόμενη πριν τελειώσει. Οι κλήσεις «στοιβάζονται» η μία πάνω στην άλλη: η **στοίβα κλήσεων**.

### Ανεβαίνει

Μόλις φτάσει στη **βάση**, οι τιμές γυρίζουν πίσω μία-μία: 1 → 2 → 6 → 24.

### Δικές της μεταβλητές

Κάθε κλήση έχει το **δικό της n** (τοπική μεταβλητή, Μάθημα 6). Γι' αυτό δεν «μπερδεύονται».

### Όριο ~1000

Η Python επιτρέπει περίπου **1000** κλήσεις τη μία μέσα στην άλλη. Μετά → RecursionError.

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ Άσκηση 3: **sum_to(n)** = n + sum_to(n − 1). Ποια είναι η βάση;

# Τι υπολογίζει; Τι εμφανίζει;

**Α**

```
def f(n):
    if n == 0:
        return 0
    return n % 10 + f(n // 10)

print(f(472))
```

**Β**

```
def g(n):
    if n > 0:
        g(n - 1)
        print(n)

g(3)
```

**ΕΚΦΩΝΗΣΗ**

Γράψε τον πίνακα των κλήσεων, όπως στο βήμα 3. Τι **υπολογίζει** το f; Με ποια **σειρά** εμφανίζει το g;

**ΛΥΣΗ**

| ΚΛΗΣΗ | ΥΠΟΛΟΓΙΖΕΙ | ΔΙΝΕΙ |
|---|---|---|
| `f(472)` | `2 + f(47)` | `13` |
| `f(47)` | `7 + f(4)` | `11` |
| `f(4)` | `4 + f(0)` | `4` |
| `f(0)` | βάση | `0` |

### Οι απαντήσεις

**Α**: το **άθροισμα των ψηφίων** (Μάθημα 1, τώρα αναδρομικά) → **13**.
**Β**: **1 2 3**! Το print είναι **μετά** την κλήση: τρέχει στο «ανέβασμα».

**ΔΟΚΙΜΑΣΕ** Στο Β άλλαξε σειρά: πρώτα **print(n)** και μετά **g(n - 1)**. Τι εμφανίζει τώρα;

# Δύο αναδρομικές κλήσεις: Fibonacci

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 4**

```
def fib(n):
    if n <= 1:                          # fib(0) = 0, fib(1) = 1
        return n
    return fib(n - 1) + fib(n - 2)

for i in range(10):
    print(fib(i), end=" ")
0 1 1 2 3 5 8 13 21 34
```

### Κάθε αριθμός

είναι το **άθροισμα των δύο προηγούμενων**: 0, 1, 1, 2, 3, 5, 8, … Ο ορισμός είναι ήδη αναδρομικός!

### Δύο κλήσεις

Το fib(n) καλεί **και** το fib(n − 1) **και** το fib(n − 2). Τα ίδια νούμερα υπολογίζονται **ξανά και ξανά**.

### Αναδρομή ή βρόχος;

Ο βρόχος είναι συνήθως **πιο γρήγορος**. Η αναδρομή κάνει τον κώδικα **πιο απλό** όταν το πρόβλημα «σπάει» σε μικρότερα ίδια: φάκελοι μέσα σε φακέλους, δέντρα, δυαδική αναζήτηση.

ΠΟΣΕΣ ΚΛΗΣΕΙΣ ΚΑΝΕΙ

| ΚΛΗΣΗ | ΣΥΝΟΛΙΚΕΣ ΚΛΗΣΕΙΣ ΤΟΥ FIB |
|---|---|
| fib(10) | 177 |
| fib(20) | 21.891 |
| fib(30) | **2.692.537** |

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ Μη δοκιμάσεις fib(30): θα αργήσει τόσο που ο έλεγχος θα το δει σαν **ατέρμονα βρόχο**.

# Ψάχνω στο μισό, κάθε φορά

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 15**

```python
def search(lst, x, lo, hi):
    if lo > hi:
        return -1                          # δεν υπάρχει
    mid = (lo + hi) // 2
    if lst[mid] == x:
        return mid                         # βρέθηκε!
    elif x < lst[mid]:
        return search(lst, x, lo, mid - 1)     # αριστερό μισό
    else:
        return search(lst, x, mid + 1, hi)     # δεξί μισό

nums = [3, 8, 12, 17, 21, 30, 42]
print(search(nums, 30, 0, len(nums) - 1))
```

```
5
```

ΨΑΧΝΩ ΤΟ 30

| LO | HI | MID | LST[MID] | ΆΡΑ |
|---|---|---|---|---|
| 0 | 6 | 3 | 17 | 30 > 17 → δεξιά |
| 4 | 6 | 5 | 30 | **βρέθηκε: 5** |

### Ταξινομημένη λίστα

Δουλεύει **μόνο** αν η λίστα είναι ταξινομημένη. Κοιτάς τη **μέση** και πετάς το μισό που δεν σε ενδιαφέρει.

### Πόσο γρήγορη

Σε **1.000.000** στοιχεία, η γραμμική αναζήτηση κάνει ως 1.000.000 συγκρίσεις· η δυαδική **περίπου 20**.

### Η βάση

Δύο βάσεις: **βρέθηκε** (return mid) ή **δεν έμεινε τίποτα** (lo > hi → return −1).

ΣΤΗΝ ΠΡΑΞΗ Έτσι ψάχνεις λέξη σε λεξικό: ανοίγεις στη μέση, και πας μπροστά ή πίσω.

# Έτοιμα εργαλεία: import math

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Ασκήσεις 5 · 6**

### ΟΛΟ ΤΟ MODULE

```
import math

print(math.sqrt(16))
print(math.pi)
print(math.ceil(4.1), math.floor(4.9))
4.0
3.141592653589793
5 4
```

### ΜΟΝΟ ΟΣΑ ΘΕΛΩ · ΜΕ ΠΑΡΑΤΣΟΥΚΛΙ

```
from math import sqrt, pi
print(sqrt(2) * pi)

import math as m
print(m.factorial(5))
4.442882938158366
120
```

### ΤΡΕΙΣ ΤΡΟΠΟΙ

| ΓΡΆΦΩ | ΜΕΤΆ ΧΡΗΣΙΜΟΠΟΙΏ |
|---|---|
| `import math` | `math.sqrt(2)` |
| `from math import sqrt` | `sqrt(2)` |
| `import math as m` | `m.sqrt(2)` |

**Module**

Ένα αρχείο .py με έτοιμες συναρτήσεις. Το **math** έρχεται **μαζί** με την Python: δεν θέλει εγκατάσταση.

**Τα import στην αρχή**

Γράφε όλα τα import στις **πρώτες γραμμές** του προγράμματος. Έτσι φαίνεται αμέσως τι χρησιμοποιεί.

**ΧΡΗΣΙΜΑ ΤΟΥ math** **sqrt** ρίζα · **pi** · **ceil / floor** στρογγύλεμα πάνω / κάτω · **gcd** ΜΚΔ · **factorial** · **sin, cos** τριγωνομετρία.

# Τυχαίοι αριθμοί και ημερομηνίες

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Ασκήσεις 7 · 8**

### RANDOM · ΑΣΚΗΣΗ 7

```
import random

print(random.randint(1, 6))           # ακέραιος από 1 έως 6
print(random.choice(["Α", "Β", "Γ"])) # ένα στοιχείο της λίστας
cards = [1, 2, 3, 4, 5]
random.shuffle(cards)                 # ανακάτεμα
print(cards)
4
Γ
[3, 1, 5, 2, 4]
```

### DATETIME · ΑΣΚΗΣΗ 8

```
from datetime import date

today = date(2026, 11, 16)
xmas = date(2026, 12, 25)
print((xmas - today).days)            # πόσες μέρες
print(xmas.weekday())                 # 0 = Δευτέρα
39
4
```

### random

Κάθε φορά βγαίνει **άλλο** αποτέλεσμα. Ζάρια, κάρτες, εχθροί σε τυχαίες θέσεις: το θέλει **κάθε παιχνίδι**.

### date(έτος, μήνας, μέρα)

Μια ημερομηνία. **date.today()** δίνει τη σημερινή.

### Αφαίρεση ημερομηνιών

**xmas − today** δίνει διάρκεια· το **.days** λέει πόσες μέρες. Η Python ξέρει πόσες μέρες έχει κάθε μήνας!

### weekday()

**0** = Δευτέρα, … **6** = Κυριακή. Με μια λίστα ονομάτων: **days[xmas.weekday()]** → "Παρασκευή".

ΘΥΜΗΣΟΥ Το **random** το είδες στο «Μάντεψε τον αριθμό» του Μαθήματος 2. Τώρα ξέρεις τι είναι το import του.

# Γράφω το δικό μου module

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 9**

ΑΡΧΕΙΟ grades_tools.py

```
def average(grades):
    """Ο μέσος όρος μιας λίστας."""
    return sum(grades) / len(grades)

def passed(grades, limit=10):
    """Οι βαθμοί από limit και πάνω."""
    result = []
    for g in grades:
        if g >= limit:
            result.append(g)
    return result
```

ΑΡΧΕΙΟ main.py · ΣΤΟΝ ΙΔΙΟ ΦΑΚΕΛΟ

```
import grades_tools

g = [12, 8, 17, 15]
print(grades_tools.average(g))
print(grades_tools.passed(g))
13.0
[12, 17, 15]
```

### Κάθε .py είναι module

Το **grades_tools.py** που έγραψες είναι module, όπως το math. Το φέρνεις με **import grades_tools** (χωρίς .py).

### Στον ίδιο φάκελο

Η Python ψάχνει πρώτα στον φάκελο του προγράμματος. Στο VS Code: **File → Open Folder** και τα δύο αρχεία μέσα.

### Γιατί;

Οι συναρτήσεις σου γράφονται **μία φορά** και τις χρησιμοποιούν **πολλά** προγράμματα. Και το main.py μένει μικρό και καθαρό.

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ Στην άσκηση 9 το **grades_tools.py** υπάρχει ήδη: εσύ γράφεις μόνο το main.py.

# if __name__ == "__main__":

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 10**

```python
# temperature.py
def c_to_f(c):
    return c * 9 / 5 + 32

def main():
    print(c_to_f(25))       # δοκιμή

if __name__ == "__main__":
    main()
```

ΤΙ ΓΙΝΕΤΑΙ ΚΑΘΕ ΦΟΡΑ

| ΠΩΣ ΤΟ ΤΡΈΧΩ | __NAME__ | ΤΡΈΧΕΙ MAIN; |
|---|---|---|
| `py temperature.py` | `"__main__"` | **ναι** → 77.0 |
| `import temperature` | `"temperature"` | όχι |

### __name__

Μια μεταβλητή που φτιάχνει η ίδια η Python. Είναι **"__main__"** μόνο στο αρχείο που **τρέχεις**.

### Το πρόβλημα που λύνει

Χωρίς το if, ένα **import temperature** θα έτρεχε και τη δοκιμή, και θα έβλεπες ένα «77.0» από το πουθενά.

### Η συνήθεια

Κάθε πρόγραμμα: **συναρτήσεις** πάνω, μια **main()**, και τελευταίες γραμμές αυτό το if.

ΔΙΑΒΑΣΕ ΤΟ ΕΤΣΙ «Αν αυτό το αρχείο είναι αυτό που **τρέχει ο χρήστης** — όχι import από άλλο — κάλεσε τη main.»

# Πακέτα και εγκατάσταση βιβλιοθηκών

### ΕΝΑ ΠΑΚΕΤΟ ΕΙΝΑΙ ΦΑΚΕΛΟΣ ΜΕ MODULES

```
game/
├── main.py
└── tools/              ← πακέτο
    ├── __init__.py
    ├── grades.py
    └── files.py
```

### ΣΤΟ main.py

```
from tools.grades import average
from tools import files
```

### ΣΤΟ ΤΕΡΜΑΤΙΚΟ ΤΟΥ VS CODE

```
> py -m venv .venv
> .venv\Scripts\activate
(.venv) > pip install numpy matplotlib
(.venv) > pip list
(.venv) > py main.py
```

#### __init__.py

Λέει στην Python «αυτός ο φάκελος είναι **πακέτο**». Μπορεί να είναι και **άδειο**.

#### pip install όνομα

Κατεβάζει μια βιβλιοθήκη από το **PyPI** (pypi.org, πάνω από 500.000 πακέτα) και την εγκαθιστά.

#### venv

Ένα ξεχωριστό «κουτί» βιβλιοθηκών **για κάθε project**. Έτσι δύο projects δεν μπερδεύουν τις εκδόσεις τους.

**ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ ΤΟΥ SITE** Δεν χρειάζεται pip: το NumPy και το Matplotlib φορτώνουν **μόνα τους** όταν γράψεις import.

# NumPy: πράξεις σε όλο τον πίνακα

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Ασκήσεις 11 · 12**

```
import numpy as np

prices = np.array([2.5, 10, 4.2])
print(prices * 1.24)              # σε ΟΛΑ μαζί, χωρίς for
[ 3.1   12.4    5.208]

grades = np.array([12, 8.5, 17, 9, 15])
print(grades.mean(), grades.max())
12.3 17.0
print(grades[grades >= 10])       # μόνο όσοι πέρασαν
[12. 17. 15.]
print((grades >= 10).sum())       # πόσοι πέρασαν
3
```

ΛΙΣΤΑ Ή ARRAY;

| ΠΡΆΞΗ | ΛΊΣΤΑ | NP.ARRAY |
|---|---|---|
| `[1, 2] * 2` | `[1, 2, 1, 2]` | `[2 4]` |
| `+ 10` | `TypeError` | `[11 12]` |

### import numpy as np

Το **np** είναι το «παρατσούκλι» που χρησιμοποιούν όλοι. Το **np.array(λίστα)** φτιάχνει πίνακα NumPy.

### Όλα μαζί

Μια πράξη σε array γίνεται σε **κάθε** στοιχείο. Και είναι **πολύ** πιο γρήγορη από ένα for, γραμμένη σε C από κάτω.

### grades >= 10

Δίνει πίνακα με True/False. Μέσα σε **[ ]** κρατά τα True· το **.sum()** μετράει πόσα True.

**ΠΟΥ ΤΟ ΒΡΙΣΚΕΙΣ** Πάνω στο NumPy στηρίζονται το pandas, το Matplotlib και σχεδόν όλη η τεχνητή νοημοσύνη στην Python.

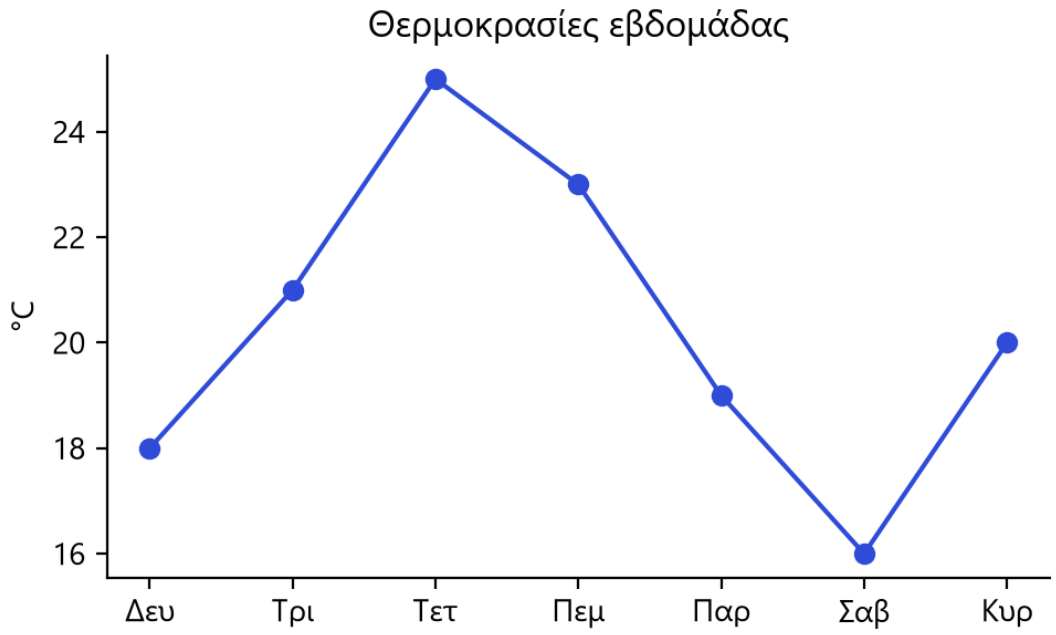
# Γράφημα γραμμής

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 13**

```python
import matplotlib.pyplot as plt

days = ["Δευ", "Τρι", "Τετ", "Πεμ", "Παρ", "Σαβ", "Κυρ"]
temps = [18, 21, 25, 23, 19, 16, 20]

plt.plot(days, temps, marker="o")
plt.title("Θερμοκρασίες εβδομάδας")
plt.ylabel("°C")
plt.show()
```




### plt.plot(x, y)

Ενώνει τα σημεία με γραμμή. Το **marker="o"** βάζει κουκκίδα σε κάθε σημείο.

### plt.show()

Ανοίγει το παράθυρο με το γράφημα. Χωρίς αυτό, στον υπολογιστή σου **δεν** θα δεις τίποτα.

ΤΙΤΛΟΙ **plt.title** τίτλος · **plt.xlabel / plt.ylabel** ονόματα αξόνων · **plt.grid()** πλέγμα.

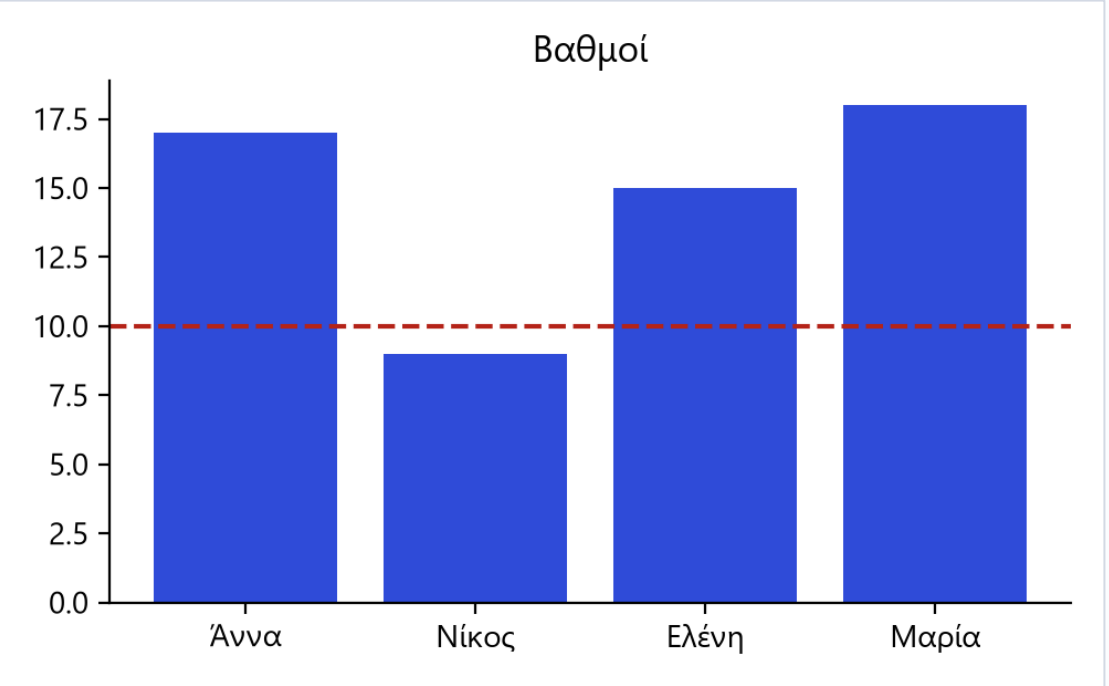
# Γράφημα ράβδων και αποθήκευση

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 14**

```python
import matplotlib.pyplot as plt

names = ["Άννα", "Νίκος", "Ελένη", "Μαρία"]
grades = [17, 9, 15, 18]

plt.bar(names, grades)
plt.axhline(10, color="red", linestyle="--")   # η βάση
plt.title("Βαθμοί")
plt.savefig("grades.png")                       # σε αρχείο εικόνας
plt.show()
```




### plt.bar(ονόματα, τιμές)

Μία **ράβδος** για κάθε στοιχείο. Για οριζόντιες: **plt.barh**.

### plt.savefig("όνομα.png")

Σώζει το γράφημα σε **εικόνα**, για εργασία ή ιστοσελίδα. Γράψ' το **πριν** το show.

ΚΑΙ ΑΛΛΑ **plt.pie** πίτα · **plt.scatter** σημεία · **plt.hist** ιστόγραμμα. Ίδια λογική: δεδομένα → γράφημα → show.

# Ποια βιβλιοθήκη για ποια δουλειά

| ΘΈΛΩ ΝΑ… | ΒΙΒΛΙΟΘΉΚΗ | ΜΙΑ ΓΕΎΣΗ |
|---|---|---|
| κάνω πράξεις σε πίνακες | `numpy` | `np.array([1, 2]) * 2` |
| διαβάσω / αναλύσω δεδομένα | `pandas` | `pd.read_csv("grades.csv")` |
| φτιάξω γραφήματα | `matplotlib` | `plt.plot(x, y)` |
| μιλήσω με ένα site / API | `requests` | `requests.get(url).json()` |
| φτιάξω ιστοσελίδα (backend) | `flask · django` | `@app.route("/")` |
| γράψω παιχνίδι 2D | `pygame` | Μαθήματα 14–15 |
| δουλέψω με Excel | `openpyxl` | Μάθημα 13 |
| επεξεργαστώ εικόνες | `pillow` | `Image.open("a.png")` |
| τεχνητή νοημοσύνη | `scikit-learn · pytorch` | `model.fit(X, y)` |

### Έρχονται μαζί

**math, random, datetime, csv, os, json, sqlite3**: χωρίς pip, μόνο import.

### Θέλουν pip

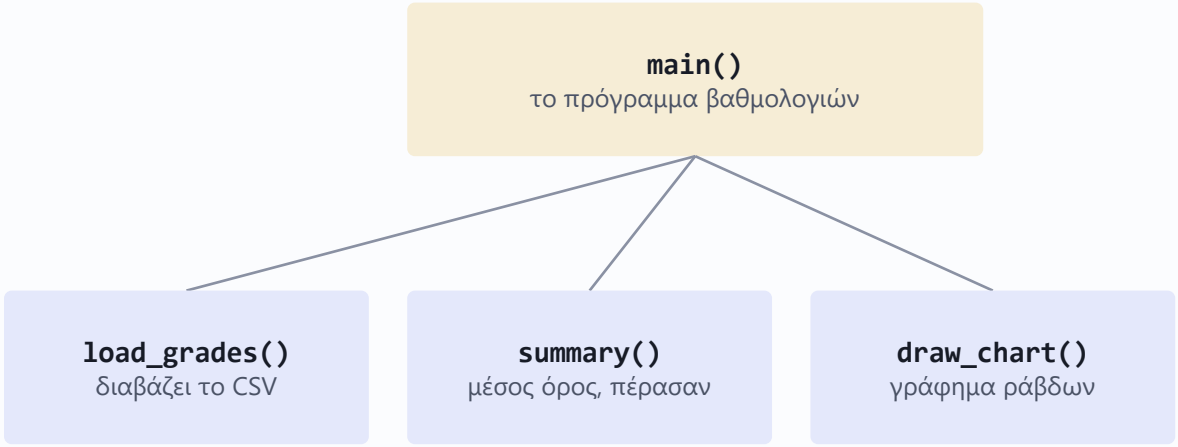
Όλες οι άλλες του πίνακα: **pip install όνομα**.

### Πώς διαλέγω

Ψάξε στο **pypi.org** και στην τεκμηρίωση. Προτίμησε ό,τι είναι **γνωστό** και **συντηρείται**.

# Σχεδιάζω το πρόγραμμα σε υποπρογράμματα

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Ασκήσεις 16 · 17**




ΤΟ main() ΔΙΑΒΑΖΕΤΑΙ ΣΑΝ ΠΕΡΙΛΗΨΗ

```
def main():
    names, grades = load_grades("grades.csv")
    summary(names, grades)
    draw_chart(names, grades)
```

### Από πάνω προς τα κάτω

Ξεκινάς από το **μεγάλο** πρόβλημα και το σπας σε **μικρότερα**. Αν ένα κομμάτι είναι ακόμα μεγάλο, το ξανασπάς.

### Ένα υποπρόγραμμα, μία δουλειά

Κάθε συνάρτηση **δοκιμάζεται χωριστά**. Αν κάτι χαλάσει, ξέρεις πού να ψάξεις.

### Και σε ομάδα

Ο καθένας γράφει **μία** συνάρτηση. Αρκεί να έχετε συμφωνήσει **τι παίρνει** και **τι επιστρέφει**.

### Τμηματικός προγραμματισμός

Αυτό ακριβώς ζητά ο οδηγός σπουδών: προγράμματα από **υποπρογράμματα**, και αυτά σε **modules**.

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ Άσκηση **16**: ο κατάλογος του Μαθήματος 5 σε υποπρογράμματα. Άσκηση **17**: αυτό το σχέδιο, ολόκληρο.

# Οι 18 ασκήσεις, βήμα-βήμα

| # | ΆΣΚΗΣΗ | ΕΠΊΠΕΔΟ | ΔΙΑΦΆΝΕΙΑ |
|---|---|---|---|
| 1 | Αντίστροφη μέτρηση | Εύκολο | 5 |
| 2 | Παραγοντικό | Εύκολο | 6 |
| 3 | Άθροισμα 1 έως n | Εύκολο | 7 |
| 4 | Fibonacci | Εύκολο | 9 |
| 5 | Κύκλος με math | Εύκολο | 11 |
| 6 | Πόσα λεωφορεία; | Εύκολο | 11 |
| 7 | Δύο ζάρια | Εύκολο | 12 |
| 8 | Μέρες ως την προθεσμία | Εύκολο | 12 |
| 9 | Το δικό μου module | Μέτριο | 13 |

| # | ΆΣΚΗΣΗ | ΕΠΊΠΕΔΟ | ΔΙΑΦΆΝΕΙΑ |
|---|---|---|---|
| 10 | Η main() και το __name__ | Μέτριο | 14 |
| 11 | NumPy: τιμές με ΦΠΑ | Μέτριο | 16 |
| 12 | NumPy: βαθμοί τάξης | Μέτριο | 16 |
| 13 | Γράφημα γραμμής | Μέτριο | 17 |
| 14 | Γράφημα ράβδων | Μέτριο | 18 |
| 15 | Δυαδική αναζήτηση | Δύσκολο | 10 |
| 16 | Κατάλογος σε υποπρογράμματα | Δύσκολο | 20, 22 |
| 17 | Γράφημα βαθμών από CSV | Ο στόχος | 20, 23 |
| 18 | Πύργοι του Ανόι | Για το σπίτι | 5–7 |

**ΣΤΟ SITE** Εργαστήριο → **Μάθημα 7**. Στα γραφήματα, το ▶ Τρέξε δείχνει την εικόνα κάτω από τον κώδικα.

# Ο κατάλογος σε υποπρογράμματα

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 16**

ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ

```python
import csv

def load_phone(filename):
    phone = {}
    try:
        with open(filename, newline="", encoding="utf-8") as f:
            for row in csv.reader(f):
                phone[row[0]] = row[1]
    except FileNotFoundError:
        pass                          # 1η φορά: κενός κατάλογος
    return phone

def save_phone(filename, phone):
    with open(filename, "w", newline="", encoding="utf-8") as f:
        csv.writer(f).writerows(phone.items())

def main():
    phone = load_phone("phone.csv")
    while True:
        choice = input("\n1.Προσθήκη 2.Όλοι 0.Έξοδος: ")
        if choice == "1":
            name = input("Όνομα: ")
            phone[name] = input("Τηλέφωνο: ")
        elif choice == "2":
            for name in sorted(phone):
                print(f"{name}: {phone[name]}")
        elif choice == "0":
            save_phone("phone.csv", phone)
            break

if __name__ == "__main__":
    main()
```

### Ίδιος κώδικας, άλλη μορφή

Ό,τι έκανε ο κατάλογος του Μαθήματος 5, τώρα σε **δύο** συναρτήσεις + **main**.

### load_phone επιστρέφει

Δεν αλλάζει καθολική μεταβλητή: **επιστρέφει** το λεξικό (Μάθημα 6, βήμα 8).

### Επόμενο βήμα

Οι δύο συναρτήσεις πάνε σε αρχείο **phone_tools.py** και το main.py γράφει **import phone_tools**.

ΑΠΟ ΠΟΙΑ ΒΗΜΑΤΑ **9** __main__ · **14** υποπρογράμματα · Μάθημα 5: **csv**, **FileNotFoundError**.

# Γράφημα βαθμολογιών από CSV

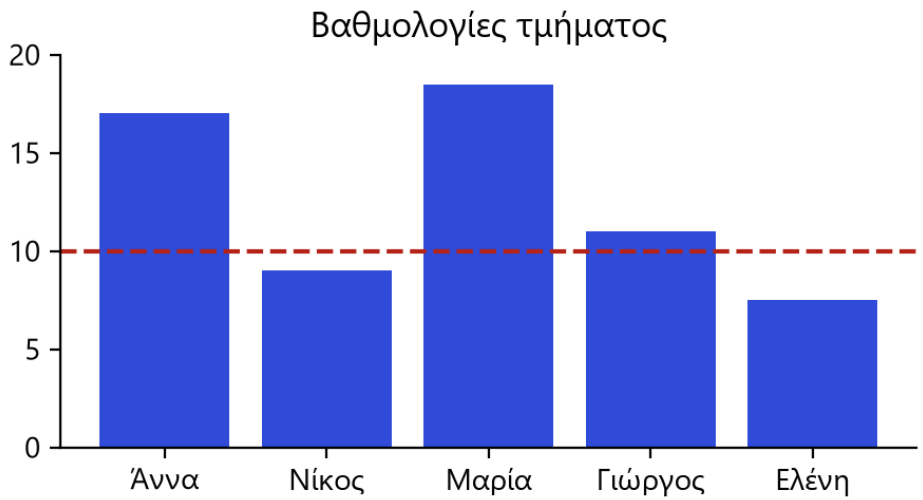
ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
**Άσκηση 17**

ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ

```python
import csv
import matplotlib.pyplot as plt

def load_grades(filename):
    names, grades = [], []
    with open(filename, newline="", encoding="utf-8") as f:
        reader = csv.reader(f)
        next(reader)                        # επικεφαλίδα
        for row in reader:
            names.append(row[0])
            grades.append(float(row[1]))
    return names, grades
def summary(names, grades):
    passed = 0
    for g in grades:
        if g >= 10:
            passed = passed + 1
    print(f"Μέσος όρος: {sum(grades) / len(grades):.2f}")
    print("Πέρασαν:", passed, "από", len(grades))
def draw_chart(names, grades):
    plt.bar(names, grades)
    plt.axhline(10, color="red", linestyle="--")
    plt.title("Βαθμολογίες τμήματος")
    plt.show()
def main():
    names, grades = load_grades("grades.csv")
    summary(names, grades)
    draw_chart(names, grades)

if __name__ == "__main__":
    main()
```




```
Μέσος όρος: 12.60
Πέρασαν: 3 από 5
```

### Το main() στο τέλος

Διαβάζεται σαν **περίληψη**: φόρτωσε → σύνοψη → γράφημα. Κάθε βήμα είναι μια **δική του** συνάρτηση.

ΑΠΟ ΠΟΙΑ ΒΗΜΑΤΑ **13** bar, axhline · **14** τρία υποπρογράμματα + main · Μάθημα 5: **csv.reader**.

# Τι κρατάμε

| ΘΈΛΩ ΝΑ… | ΓΡΆΦΩ |
|---|---|
| γράψω αναδρομή | `βάση: return … · αλλιώς: f(μικρότερο)` |
| φέρω ένα module | `import math · from math import sqrt` |
| δώσω παρατσούκλι | `import numpy as np` |
| τυχαίο ακέραιο | `random.randint(1, 6)` |
| μέρες ανάμεσα σε ημερομηνίες | `(d2 - d1).days` |
| τρέξω κώδικα μόνο αν τρέχει το αρχείο | `if __name__ == "__main__":` |
| εγκαταστήσω βιβλιοθήκη | `pip install numpy` |
| πράξη σε όλο τον πίνακα | `np.array(λ) * 2 · a[a >= 10]` |
| γράφημα | `plt.plot · plt.bar · plt.show()` |

### Μικρά κομμάτια

Η **αναδρομή** σπάει το πρόβλημα σε μικρότερα ίδια. Τα **modules** σπάνε το πρόγραμμα σε αρχεία.

### Επόμενη φορά · Δευ 23/11

**Συμβολοσειρές**: split, join, replace, f-strings με στοίχιση, και τα ελληνικά σε UTF-8.

**ΓΙΑ ΤΟ ΣΠΙΤΙ** Άσκηση **18** στο εργαστήριο: οι Πύργοι του Ανόι, η πιο διάσημη αναδρομή (βήματα 1–3).